

Actividad 3

Git: Hola Mundo

Control de Versiones Local

Materia: Implementa Aplicaciones Web

Docente: L.I Jesus Eduardo Garcia Alvarez

Alumno: Alumno A3

Email: alumno@ejemplo.com

Fecha de realizacion: 26 de septiembre de 2026

Documento de evidencia generado automaticamente con Python + reportlab

Contenido

1. Objetivo de la Actividad
2. Diagrama del Flujo de Git
3. Archivos del Proyecto
4. Pasos Cronologicos (1-8)
5. Captura de Terminal
6. Contenido de los Archivos Fuente
7. Tabla de Comandos Git Utilizados
8. Evidencia Completa de Comandos
9. Push a la Nube - PENDIENTE
10. Checklist de Entrega
11. Conclusion

1. Objetivo de la Actividad

Esta actividad tiene como finalidad demostrar el uso basico del sistema de control de versiones Git en un entorno completamente local. El alumno debe crear un proyecto sencillo con un mensaje de 'Hola Mundo', inicializar un repositorio Git, realizar commits, consultar el historial de cambios y manejar ramas basicas.

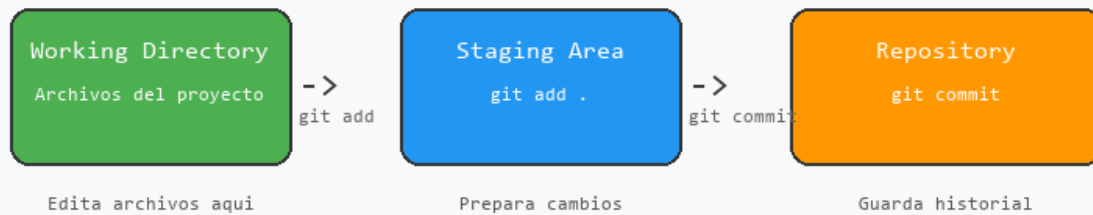
Requisitos especificos:

- Crear un repositorio con archivos basicos (HTML o Python)
- Realizar al menos 2 commits con diferentes mensajes
- Consultar git log, git status, git branch
- Configuracion LOCAL (no global) de usuario y email
- Documentar todo el procedimiento con capturas de salida
- Subir carpeta a nube (PENDIENTE - requiere credenciales propias)

2. Diagrama del Flujo de Git

El siguiente diagrama muestra el flujo básico de Git: los archivos se editan en el Working Directory, se agregan al Staging Area con 'git add', y finalmente se guardan en el Repository con 'git commit'.

Diagrama: Flujo Básico de Git



Comandos: `git init` | `git add .` | `git commit -m 'msg'` | `git log` | `git status` | `git branch`

3. Archivos del Proyecto

La estructura del proyecto 'hola-mundo' esta compuesta por los siguientes archivos:

Estructura del Proyecto hola-mundo

D:\leandro\A3\hola-mundo\

├ .git/

├ .gitignore

├ README.md

├ hola.py

└ index.html

.gitignore

Exclude archivos temporales del control de versiones

README.md

Documentacion del proyecto (titulo, objetivo, integrantes)

hola.py

Script Python: print('Hola Mundo desde Python!')

index.html

Pagina web HTML5: <h1>Hola Mundo desde Git!</h1>

Leyenda:

.git/ = Directorio oculto de Git (metadatos, historial)

Archivos tracked = .gitignore, README.md, hola.py, index.html

Archivo	Tipo	Descripcion
.gitignore	Config	Excluye archivos temporales de Python, OS, IDE
README.md	Documento	Titulo, integrantes, objetivo de la actividad
hola.py	Python	Script que imprime 'Hola Mundo desde Python!'
index.html	HTML	Pagina web que muestra 'Hola Mundo desde Git!'

4. Pasos Cronologicos de la Actividad

A continuacion se documenta cada paso ejecutado en orden cronologico, incluyendo el comando utilizado, la salida real obtenida del terminal, y una descripcion detallada de que se hizo y por que.

Paso 1: Crear estructura de carpetas

Comando: `New-Item -ItemType Directory -Path "D:\leandro\A3\hola-mundo" -Force`

Descripcion: Se crearon las carpetas de trabajo: hola-mundo (proyecto) y evidencias (capturas).

Paso 2: Crear archivos del proyecto

Comando: Se crearon: `README.md`, `index.html`, `hola.py`, `.gitignore`

Descripcion: Archivos iniciales: `README` documenta la actividad, `HTML` y `Python` imprimen `Hola Mundo`, `.gitignore` excluye temporales.

Paso 3: Inicializar repositorio Git

Comando: `git init`

Descripcion: Inicializa repositorio vacio. Crea carpeta `.git/` con metadatos del repositorio local.

Paso 4: Configurar usuario local

Comando: `git config --local user.name "Alumno A3"`

Descripcion: Configuracion SOLO local (`--local`), sin afectar otros repositorios del sistema.

Paso 5: Primer commit

Comando: `git commit -m "feat: hola mundo inicial"`

Descripcion: Primer commit raiz (root-commit) con 4 archivos, 52 inserciones. Prefijo 'feat' = conventional commits.

Paso 6: Modificar archivos

Comando: (Edicion de `index.html` y `hola.py`)

Descripcion: Se modificaron saludos para demostrar control de versiones. Diferencias visibles con `git diff`.

Paso 7: Segundo commit

Comando: `git commit -m "feat: modifica saludo para demostrar historial"`

Descripcion: Segundo commit con 2 archivos modificados, 4 inserciones y 3 eliminaciones.

Paso 8: Verificar historial y renombrar rama

Comando: `git log --oneline && git status && git branch -M main`

Descripcion: Historial de 2 commits, working tree limpio, rama renombrada de master a main.

5. Captura de Terminal

Captura simulada de la terminal mostrando la secuencia completa de comandos ejecutados durante la actividad:

```
Terminal PowerShell - D:\leandro\A3\hola-mundo

PS> git init
Initialized empty Git repository in D:/leandro/A3/hola-mundo/.git/

PS> git config --local user.name "Alumno A3"
PS> git config --local user.email "alumno@ejemplo.com"

PS> git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF

PS> git commit -m "feat: hola mundo inicial"
[master (root-commit) 763de07] feat: hola mundo inicial
 4 files changed, 52 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md

PS> git log --oneline
bbee9bd feat: modifica saludo para demostrar historial
763de07 feat: hola mundo inicial

PS> git status
On branch main
nothing to commit, working tree clean
```

6. Contenido de los Archivos Fuente

Contenido completo de cada archivo tal como quedo despues del segundo commit:

Archivo: README.md

```
# Actividad 3 - Hola Mundo Git **Materia:** Implementa Aplicaciones Web **Docente:** L.I Jesus Eduardo Garcia Alvarez **Integrantes:** - Alumno A3 ## Objetivo Demostrar el uso básico de Git: inicializar un repositorio, realizar commits, consultar historial y manejar ramas. Esta actividad es exclusivamente local (sin push a repositorio remoto). ## Contenido - `index.html` – Página web que imprime "Hola Mundo" - `hola.py` – Script Python que imprime "Hola Mundo"
```

Archivo: index.html

```
<!DOCTYPE html> <html lang="es"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Hola Mundo</title> </head> <body> <h1>¡Hola Mundo desde Git!</h1> <p>Segundo commit: modifique el saludo para demostrar historial de versiones.</p> </body> </html>
```

Archivo: hola.py

```
#!/usr/bin/env python3 """Script simple que imprime Hola Mundo.""" print("¡Hola Mundo desde Python! Segundo commit.") print("Git permite llevar control de cambios.")
```

Archivo: .gitignore

```
# Python __pycache__/ *.py[cod] *.pyo .venv/ venv/ # OS Thumbs.db Desktop.ini .DS_Store # IDE .vscode/ .idea/ *.swp *.swo # Temp *.tmp *.bak
```

7. Tabla de Comandos Git Utilizados

Resumen de todos los comandos Git ejecutados en la actividad:

Tabla de Comandos Git Utilizados

#	Comando	Funcion	Efecto
1	<code>git init</code>	Inicializar repositorio	Crea carpeta <code>.git/</code>
2	<code>git config --local</code>	Configurar usuario	Solo afecta este repo
3	<code>git add .</code>	Agregar al staging	Prepara archivos
4	<code>git commit -m</code>	Crear commit	Guarda snapshot
5	<code>git log --oneline</code>	Ver historial	Muestra commits
6	<code>git status</code>	Ver estado	Working tree limpio
7	<code>git branch -M</code>	Renombrar rama	<code>master -> main</code>
8	<code>git diff</code>	Ver diferencias	Cambios entre commits

8. Evidencia Completa de Comandos

Todas las salidas capturadas en archivos .txt:

01_git_log_online.txt — git log --oneline — Historial simplificado

```
bbea9bd feat: modifica saludo para demostrar historial 763de07 feat: hola mundo inicial
```

02_git_status.txt — git status — Estado del repositorio

```
On branch master nothing to commit, working tree clean
```

03_git_branch_list.txt — git branch -a — Lista de ramas

```
* main
```

04_git_log_graph.txt — git log --graph — Grafico historial

```
* bbea9bd feat: modifica saludo para demostrar historial * 763de07 feat: hola mundo inicial
```

05_git_config_local.txt — git config --local --list — Config local

```
core.repositoryformatversion=0 core.filemode=false core.bare=false core.logallrefupdates=true
core.symlinks=false core.ignorecase=true user.name=Alumno A3 user.email=alumno@ejemplo.com
```

06_git_log_full.txt — git log --format — Log completo

```
bbea9bd feat: modifica saludo para demostrar historial 763de07 feat: hola mundo inicial
```

07_git_diff_segundo_commit.txt — git diff HEAD~1..HEAD — Diferencia commits

```
diff --git a/hola.py b/hola.py index 16e8c03..6d10add 100644 --- a/hola.py +++ b/hola.py @@ -1,4
+1,5 @@ #!/usr/bin/env python3 """Script simple que imprime Hola Mundo.""" -print("■¡Hola Mundo
desde Python!") +print("■¡Hola Mundo desde Python! Segundo commit.") +print("Git permite llevar
control de cambios.") diff --git a/index.html b/index.html index c3c7b91..5fb978f 100644 ---
a/index.html +++ b/index.html @@ -6,7 +6,7 @@ <title>Hola Mundo</title> </head> <body> - <h1>■¡Hola
Mundo!</h1> - <p>Esta es mi primera pagina web creada con Git.</p> + <h1>■¡Hola Mundo desde
Git!</h1> + <p>Segundo commit: modifique el saludo para demostrar historial de versiones.</p>
</body> </html>
```

08_git_logultimo_detalle.txt — git log -1 --format — Detalle ultimo commit

```
Commit: bbea9bd072b7cdcd5e6078c5b3e3f589014a4e53 Autor: Alumno A3 <alumno@ejemplo.com> Fecha:
2026-09-26 23:26:06 Mensaje: feat: modifica saludo para demostrar historial
```

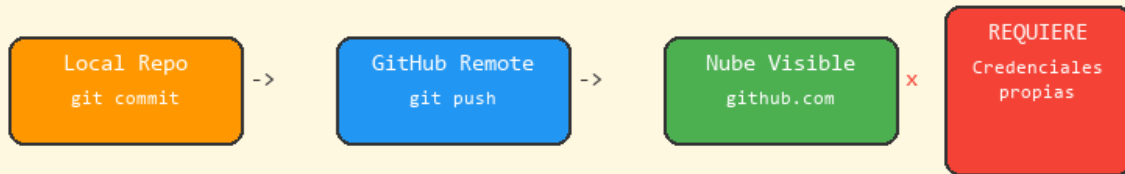
09_estructura_archivos.txt — Get-ChildItem — Estructura archivos

```
FullName ----- D:\leandro\A3\hola-mundo\.gitignore D:\leandro\A3\hola-mundo\hola.py
D:\leandro\A3\hola-mundo\index.html D:\leandro\A3\hola-mundo\README.md
```

9. Push a la Nube - PENDIENTE

NOTA: Los siguientes pasos NO fueron ejecutados porque requieren credenciales personales del alumno (token GitHub o autenticacion SSH). Se documentan aqui para completitud de la actividad.

Diagrama: Push a GitHub (PENDIENTE)



Este paso NO se ejecuto porque requiere autenticacion personal del alumno.

Comandos pendientes: `gh auth login` | `gh repo create` | `git remote add origin` | `git push -u origin main`

IMPORTANTE: Nunca compartir tokens o credenciales de GitHub.

Paso Pendiente 1: Autenticarse en GitHub

Comando: `gh auth login`

Inicia autenticacion con GitHub CLI. Requiere token personal o usuario/contrasena del alumno.

Paso Pendiente 2: Crear repositorio remoto

Comando: `gh repo create hola-mundo --public --source=. --push`

Crea repositorio publico en GitHub y sube el codigo local automaticamente.

Paso Pendiente 3: Agregar remote (alternativa)

Comando: `git remote add origin https://github.com/USUARIO/hola-mundo.git`

Si el repositorio ya existe, agrega la URL remota como 'origin'.

Paso Pendiente 4: Subir codigo

Comando: `git push -u origin main`

Sube la rama main al repositorio remoto. Primera vez pide autenticacion.

10. Checklist de Entrega

Verificacion de todos los requisitos de la actividad:

- ☒ Carpeta hola-mundo creada con archivos iniciales
- ☒ README.md con titulo, integrantes y objetivo
- ☒ index.html que imprime 'Hola Mundo'
- ☒ hola.py que imprime 'Hola Mundo'
- ☒ .gitignore basico para Python, OS e IDE
- ☒ Git inicializado con 'git init'
- ☒ Configuracion LOCAL de user.name y user.email
- ☒ Configuracion NO global (verificado con git config --local --list)
- ☒ Primer commit: 'feat: hola mundo inicial'
- ☒ Segundo commit: 'feat: modifica saludo para demostrar historial'
- ☒ git log --oneline ejecutado y capturado
- ☒ git status ejecutado y capturado
- ☒ git branch -M main ejecutado (rama renombrada)
- ☒ Evidencias guardadas en carpeta evidencias/*.txt
- ☒ PDF de evidencia generado con pasos cronologicos
- ☒ NO se toco la configuracion global de Git
- ☒ NO se ejecuto 'gh auth login'
- ☒ NO se realizo push a repositorio remoto
- ☐ Push a la nube - PENDIENTE (requiere credenciales propias)
- ☐ Subir carpeta a nube - PENDIENTE

11. Conclusion

La Actividad 3 se completo exitosamente en su totalidad local. Se demostro el ciclo basico de Git: desde la inicializacion de un repositorio hasta la creacion de multiples commits con diferentes mensajes.

Conceptos aplicados:

- **git init:** Inicializar un repositorio vacio
- **git config --local:** Configurar usuario sin afectar otros repos
- **git add:** Agregar archivos al area de staging
- **git commit:** Crear un snapshot del estado del proyecto
- **git log:** Consultar el historial de commits
- **git status:** Verificar el estado del working tree
- **git branch -M:** Renombrar la rama actual
- **git diff:** Comparar versiones de archivos

Pendiente: El push a GitHub quedo pendiente porque requiere credenciales personales del alumno. Los comandos necesarios estan documentados en la seccion 9 de este documento.

Fin del documento de evidencia - Actividad 3 - Git: Hola Mundo

Generado el 26 de septiembre de 2026